

| Modules de Node JS

| Core Modules

- La modularité est une technique de conception de logiciels dans laquelle un programme comporte des parties distinctes, chacune fournissant un seul élément de la fonctionnalité globale
 - Ces modules distincts s'unissent pour former un tout cohérent.
 - La modularité est essentielle pour créer des programmes évolutifs qui intègrent des bibliothèques et des cadres et séparent le programme en morceaux gérables.
 - Essentiellement, un module est une collection de code située dans un fichier. Au lieu d'avoir un programme entier situé dans un seul fichier, le code est organisé en fichiers séparés en fonction des problèmes auxquels ils répondent.
 - Ces fichiers peuvent ensuite être inclus dans d'autres fichiers en utilisant la fonction `require()`.
 - le module `EVENTS` est un module de base de Node.js qui fournit des fonctions clés pour travailler avec, eh bien... des événements
-

| Process Module

- Le `process.memoryUsage()` renvoie des informations sur les demandes de processeur du processus en cours. Il renvoie une propriété qui ressemble à ceci :

```
console.log(process.memoryUsage())
{
  rss: 26247168,
  heapTotal: 5767168,
  heapUsed: 3573032,
  external: 8772,
  arrayBuffers: 11214
}
```

- La propriété `process.argv` contient un tableau de valeurs de ligne de commande fournies lorsque le processus en cours a été lancé.
 - Le premier élément du tableau est le chemin absolu vers Node, qui a exécuté le processus.
 - Le deuxième élément du tableau est le chemin d'accès au fichier en cours d'exécution.

- Les éléments suivants seront tous les arguments de ligne de commande fournis lorsque le processus a été lancé.

```
console.log(process.argv[2])
node app.js test javascript
//resultat: test
```

- La propriété `process.env` est un objet qui stocke et contrôle les informations sur l'environnement dans lequel le processus est en cours d'exécution. Par exemple, l'objet `process.env` contient une propriété `PWD` qui contient une chaîne avec le répertoire dans lequel se trouve le processus actuel.

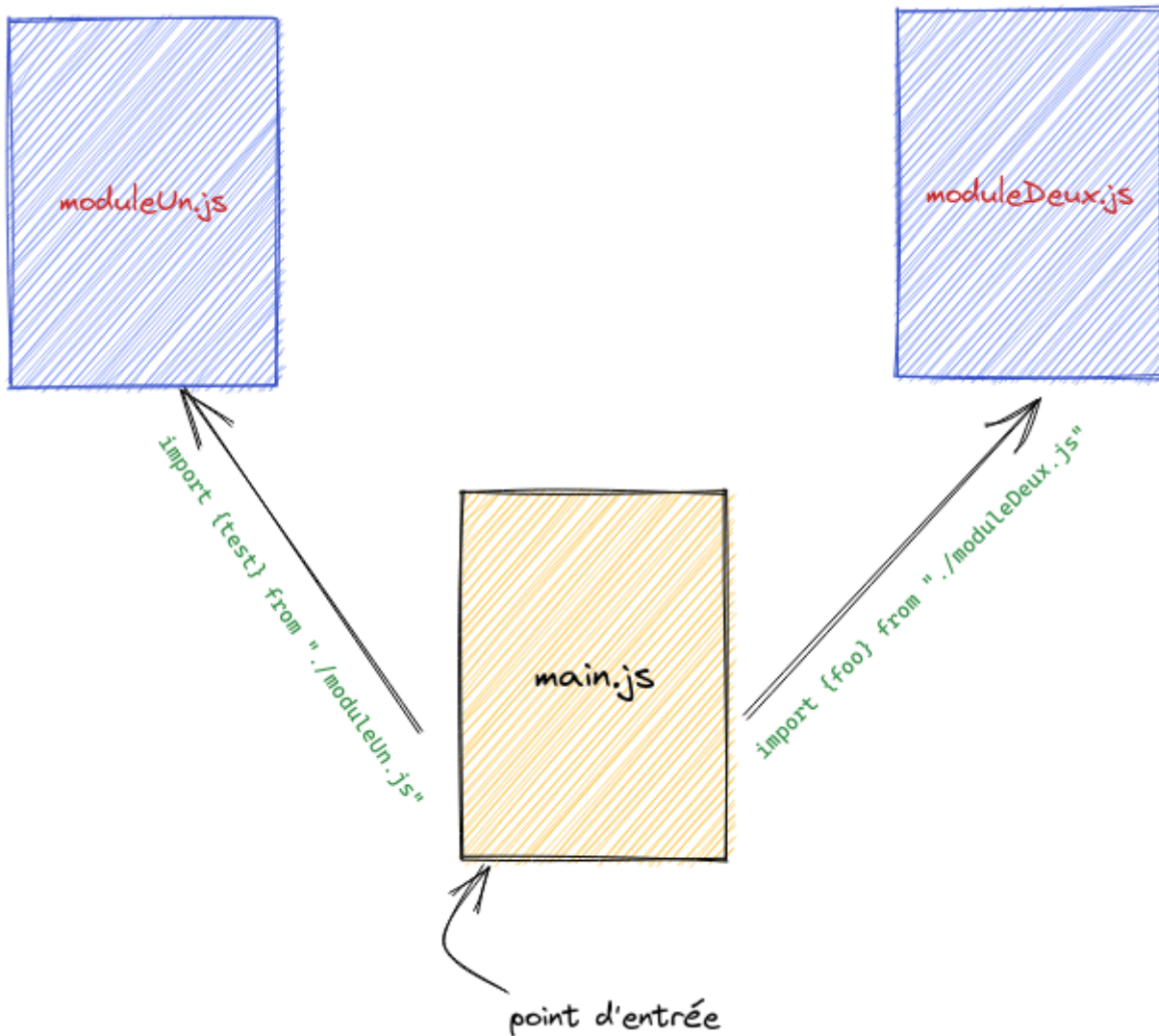
OS Module

- Lors du développement ou du débogage d'une application, il peut être utile d'avoir des informations sur l'ordinateur, le système d'exploitation et le réseau sur lequel le programme s'exécute. Avant Node, ces informations ne pouvaient pas être récupérées à l'aide de JavaScript car le langage était confiné au navigateur. Cependant, Node.js est un runtime JavaScript, ce qui signifie qu'il peut exécuter du code en dehors du navigateur, et nous pouvons accéder à une grande partie de ces informations via le module `os`.

MODULES

Les modules sont des morceaux de code réutilisables dans un fichier qui peuvent être exportés puis importés pour être utilisés dans un autre fichier. Un programme modulaire est un programme dont les composants peuvent être séparés, utilisés individuellement et

recombinés pour créer un système complexe.



| Note: Le mot « module » et « fichier » sont souvent utilisés de manière interchangeable

- Pourquoi utiliser les modules?

En isolant le code dans des fichiers séparés, appelés modules, vous pouvez :

- trouvez, corrigez et déboguez le code plus facilement.
- réutilisez et recyclez la logique définie dans différentes parties de votre application.
- garder les informations privées et protégées des autres modules.
- éviter la pollution et les collisions de noms potentielles, en sélectionnant avec précaution les variables et le comportement que nous chargeons dans un programme.

-> Supposons que vous vouliez écrire un programme simple qui afficherait le point de congélation et le point d'ébullition de l'eau en Fahrenheit. Ce programme peut être comme ça:

```
/* eauTemp.js */  
function celsiusToFahrenheit(celsius) {  
  return celsius * 1.8 + 32;  
}
```

```

const freezingPointC = 0;
const boilingPointC = 100;

const freezingPointF = celsiusToFahrenheit(freezingPointC);
const boilingPointF = celsiusToFahrenheit(boilingPointC);

console.log(`Le point de congélation de l'eau en Fahrenheit est
${freezingPointF}`);
console.log(`le point d'ébullition de l'eau en Fahrenheit est
${boilingPointF}`);

```

Le résultat:

```

$ node eauTemp.js
The freezing point of water in Fahrenheit is 32
The boiling point of water in Fahrenheit is 212

```

-> Maintenant, nous allons décider l'input et la température.. Nous allons écrire une application et lui donner la température qu'on veut ... elle sera comme ça:

```

/* celsius-to-fahrenheit.js */
function celsiusToFahrenheit(celsius) {
    return celsius * 1.8 + 32;
}

const celsiusInput = process.argv[2]; // Obtenir le 3ème input de la
liste d'arguments
const fahrenheitValue = celsiusToFahrenheit(celsiusInput);

console.log(`${celsiusInput} degrés Celsius = ${fahrenheitValue} degrés
Fahrenheit`);

```

Le résultat:

```

$ node celsius-to-fahrenheit.js 13
13 degrés Celsius = 55 degrés Fahrenheit

```

| module.exports

| EXO

- Shape Area Calculate - app.js file

```

/*fichier: app.js*/

const radius = 5;
const side = 10;

// Option 1: importer the entire shape-area.js module here.
// const shapeArea = require('./shape-area.js')

// Option 2 : importer circleArea et squareArea avec déstructuration
d'objet
const { circleArea, squareArea } = require("./shape-area.js")

// utiliser les méthodes importées de .circleArea() et .squareArea() ici

const areaOfCircle = circleArea(radius)
const areaOfSquare = squareArea(side)

console.log(areaOfCircle);
console.log(areaOfSquare);

```

- shapeArea.js file

```

// const PI = Math.PI;

// Déclarer circleArea() ici
// calculer le cercle = PI * radius * radius

// Déclarer squareArea() ici
//calculer le square = side * side

// Définir et exporter circleArea() et squareArea() en-dessous

-----

//Solution

const PI = Math.PI;

function circleArea(radius){
  return PI * radius * radius
}

function squareArea(side){
  return side * side
}

// Define and export circleArea() and squareArea() below

```

```
module.exports.circleArea = circleArea  
module.exports.squareArea = squareArea
```